Drug Administration Analysis in Excel

This guide replicates the Python notebook logic in Excel. The goal is to classify each dose row by drug subtype and calculate total ordered amounts, using a four-step fallback to resolve the dose for each row.

What is being measured: stock drawn, not drug delivered. A dose pulled from stock is consumed by the hospital whether it is fully given, partly given, or discarded. The ordered amount is therefore the quantity of interest, and the administered columns must never be allowed to *reduce* a total. They only fill a gap where nothing else was recorded.

Requires Excel 365. The formulas use `XLOOKUP`, `XMATCH`, `IFS`, `TEXTBEFORE`, and `TEXTAFTER`. `XLOOKUP`, `XMATCH`, and `IFS` are also in Excel 2021, but `TEXTBEFORE` and `TEXTAFTER` are not, so Step 4 needs 365.

> **Working in Google Sheets instead?** Three formulas need a change, all of them in Steps 3 and 4. You can find their alternatives within this guide!

The design principle throughout: **drug names, units, and conversion factors live in tables, not in formulas.** Adding a drug or a unit means adding a row to the `Reference` sheet. You should never have to edit a formula to onboard new data. If you get new data with new drug types and units with their own conversions, editing the `Reference` sheet should be all that is required.

> **NOTE:** While this guide is written in regard to the full medication administrations file, this guide will still function just fine for the apheresis file. The function setup will be complete overkill for that file, but writing the functions for just the cases in the apheresis file would break the full medication administrations file.

Column Reference

After importing the CSV (see Step 1), the columns are:

Column	Name
A	Start Date
B	End Date
C	Slices by Component Simple Generic Name
D	Order Name
E	Ordered Dose Amount
F	Ordered Dose Unit
G	Administration Instant
H	Administered Dose Amount (**last resort only**)
I	Administered Dose Unit (**last resort only**)

> **Important:** Columns H and I are mainly checker columns. Dose logic uses D, E and F first, then the Order Name. H and I are read **only** at step 3 of the fallback, for the few rows that record no ordered amount and carry no dose in the name (see Step 5). A row resolved that way is a **lower bound** on the stock drawn, because a part-used vial still left the shelf whole.

Helper columns you will add run from **J to P**:

Column	Header	Purpose
J	Drug Type	Which drug subtype this row is
K	Name Unit	Unit found at the end of the Order Name
L	Name Amount	Number that precedes that unit
M	Dose Source	Which fallback step resolved this row
N	Resolved Unit	The unit actually used
O	Resolved Amount	The amount actually used
P	Converted Amount	Resolved Amount converted to the drug's target unit

Step 1: Open and Prepare the Data

Open the data in Excel and name the sheet something descriptive, such as `Data` or `Medication Data` to keep yourself organized. Delete the first few rows so that the column headers (`Start Date`, `End Date`, etc.) are in **row 1** and data begins in **row 2**. You can easily set up this analysis without removing the metadata rows, but the formulas below are written as if the headers are on **row 1**, so keep that in mind if you are copying/writing these formulas.

Step 2: Set Up the Reference Sheet

Create a new sheet named `Reference`. It holds four small tables. Everything the formulas need to know about drugs, units, and defaults lives here.

Leave one blank column between tables so nothing collides.

Table 1: Drug Keywords (A1:B12)

Each row maps a keyword to a drug type. **Order matters.** The first keyword found in the Order Name wins, so keep the sequence below.

keyword	drugtype
albumin human 5 %	Albumin 5%
albumin human 25 %	Albumin 25%
balfaxar	Balfaxar (4F-PCC)
prothrombin complex human-lans	Balfaxar (4F-PCC)

keyword	drugtype
kcentra	Kcentra (4F-PCC)
antithrombin	Antithrombin III (Thrombate)
alphanate	Antihemophilic Factor-VWF (Alphanate)
rahf-pfm	Antihemophilic Factor rAHF-PFM (Advate)
advate	Antihemophilic Factor rAHF-PFM (Advate)
coagulation factor viia	Coagulation Factor VIIa (NovoSeven)
coagulation factor ix	Coagulation Factor IX (Benefix)

Two drugs have two keywords each, Balfaxar and Advate, which is why this table has 11 rows for 9 drug types. To add a new alias, add a row.

Define these names (**Formulas → Define Name**):

- `kw_text` → `Reference!$A$2:$A$12`
- `kw_type` → `Reference!$B$2:$B$12`

The screenshots folder shows where to define these ranges. If you would rather not use names, replace every mention of one with the range itself (for example, replace `kw_text` with `Reference!$A$2:$A$12` in the formula).

Table 2: Unit Conversions (D1:G16)

Column G is a **key column** that joins the drug type and unit into one lookup value. This is what lets every later formula use a plain `XLOOKUP` instead of an array formula.

In **G2** enter `=D2&"|"&E2` and fill down. Do not type the keys by hand.

The formula has only two moving parts, but the later formulas depend on it, so it is worth setting them out:

Piece	On its own	What it contributes here
`D2` , `E2`	The drug type and source unit on this row.	The two things that together identify a conversion factor. Neither is unique alone: `Units` appears against five drugs, and `Alphanate` appears against three units.
`&`	Joins two pieces of text into one.	Collapses that two-part identity into a single value, so it can be matched with one lookup instead of two coordinated ones.
the pipe separator	The single literal character sitting between the two joined values.	Chosen because it never occurs in a drug name or a unit. Without a separator, `Albumin 5%` + `g` and `Albumin 5` + `%g` would collapse into the same key.

Matching on two columns at once normally forces an array formula. Doing the join once, here, in a real column of cells, means every lookup later is an ordinary one.

drugtype	sourceunit	factor	key *(formula)*
Albumin 5%	g	1	*(formula above)*
Albumin 5%	mL	0.05	*(fill down)*
Albumin 5%	L	50	*(fill down)*
Albumin 25%	g	1	*(fill down)*

drugtype	sourceunit	factor	key *(formula)*
Albumin 25%	mL	0.25	*(fill down)*
Balfaxar (4F-PCC)	Units	1	*(fill down)*
Kcentra (4F-PCC)	Units	1	*(fill down)*
Antithrombin III (Thrombate)	Units	1	*(fill down)*
Antihemophilic Factor-VWF (Alphanate)	VWF:RCo Units	1	*(fill down)*
Antihemophilic Factor-VWF (Alphanate)	Int'l Units	1	*(fill down)*
Antihemophilic Factor-VWF (Alphanate)	Units	1	*(fill down)*
Antihemophilic Factor rAHF-PFM (Advate)	Units	1	*(fill down)*
Coagulation Factor VIIa (NovoSeven)	mg	1	*(fill down)*
Coagulation Factor VIIa (NovoSeven)	mcg	0.001	*(fill down)*
Coagulation Factor IX (Benefix)	Units	1	*(fill down)*

That is 15 conversion rows, so the data occupies rows 2 to 16. Define:

- `conv_key` → `Reference!$G$2:$G$16`
- `conv_factor` → `Reference!$F$2:$F$16`

> Check the row numbers if you add or remove a conversion. A range that stops short of the last row silently drops that drug, and the only sign of it is `"Unit not found"` in column P.

Table 3: Default Doses and Administered Opt-In (I1:L10)

Column L is a per-drug switch controlling whether the **Administered** fallback (step 3 of Step 5) may be used for that drug. It lives here, not in the formula, so the exception stays in a table.

drugtype	defaultamount	defaultunit	useadmin
Albumin 5%	12.5	g	no
Albumin 25%	12.5	g	no
Balfaxar (4F-PCC)	*(leave blank)*	Units	yes
Kcentra (4F-PCC)	*(leave blank)*	Units	yes
Antithrombin III (Thrombate)	*(leave blank)*	Units	yes
Antihemophilic Factor-VWF (Alphanate)	*(leave blank)*	Units	yes
Antihemophilic Factor rAHF-PFM (Advate)	*(leave blank)*	Units	yes
Coagulation Factor VIIa (NovoSeven)	*(leave blank)*	mg	yes
Coagulation Factor IX (Benefix)	*(leave blank)*	Units	yes

Define:

- `def_type` → `Reference!$I$2:$I$10`
- `def_amt` → `Reference!$J$2:$J$10`
- `def_unit` → `Reference!$K$2:$K$10`

- `def_admin` → `Reference!$L$2:$L$10`

> **Why albumin is 12.5 and the concentrates are blank.** Albumin is dispensed in whole bottles, so one bottle, 12.5 g, is a real stock draw, and it is the right answer for the albumin rows whose Order Name carries no size. The concentrates are the opposite case. There is no sensible default for them, so a concentrate row reaching `Default` is a genuine gap in the record. Leaving the amount blank keeps that row at 0 and visible.
>
> **Do not put a placeholder such as `1` here.** It makes a row look resolved while feeding a fabricated number into the total, for example two Kcentra rows summing to `2 Units` when the real draw was `4,000 Units`.

> **Why albumin sets `useadmin` to `no`.** A partial administered volume would *understate* the draw: 85 mL charted against a 12.5 g bottle is still a 12.5 g draw. For albumin the bottle default is the closer estimate, so it skips step 3 and goes straight to the default. The concentrates have no such default, so for them a recorded administered amount is better than nothing.

Table 4: Units Found in Order Names (M1:M8)

The units that can appear at the end of an Order Name, **longest first**. The order is what makes multi-word units work: `VWF:RCo Units` must be tested before `Units`, or the match would stop at the last word and lose the prefix. Likewise `mcg` before `mg` before `g`.

nameunit
VWF:RCo Units
Int'l Units
Units
mcg
mg
mL
g

Define:

- `name_units` → `Reference!$M$2:$M$8`

Step 3: Drug Type Classification (Column J)

Add a header `Drug Type` in **J1**. In **J2**:

```
=IFNA(XLOOKUP(TRUE, ISNUMBER(SEARCH(kw_text, D2)), kw_type), "Unmatched")
```

In **Google Sheets**, wrap the array-producing part in `ARRAYFORMULA` so all 11 keywords are tested instead of just the first:

```
=IFNA(XLOOKUP(TRUE, ARRAYFORMULA(ISNUMBER(SEARCH(kw_text, D2))), kw_type), "Unmatched")
```

Fill down for all rows.

Piece by piece:

Piece	On its own	What it contributes here
`SEARCH(kw_text, D2)`	Finds where one string starts inside another, case-insensitively. Returns a character position, or `#VALUE!` if the string isn't there.	`kw_text` is 11 cells, so this performs 11 searches at once and returns 11 answers: a position for every keyword present in the Order Name, an error for every one that isn't.
`ISNUMBER(...)`	Returns `TRUE` for a number and `FALSE` for anything else, **including errors**.	Flattens that mixed array into 11 clean flags meaning *did this keyword hit?* It is also what absorbs the `#VALUE!` results, so they never reach the outer functions.
`XLOOKUP(TRUE, <flags>, kw_type)`	Searches a range for a value and returns the entry in the same position of a second range.	Finds the **first** `TRUE` and returns the drug type beside it. First-match is what makes Table 1's row order the precedence order.
`IFNA(..., "Unmatched")`	Supplies a fallback when the expression inside it returns `#N/A`, **and only** `#N/A`. Any other error passes straight through.	`XLOOKUP` returns `#N/A` when no flag was `TRUE`, and this turns that into the readable label `"Unmatched"`.

Together: the two inner functions convert "*does this name contain any of my keywords?*" into a list of yes/no answers. The two outer functions take the first yes and name it, or say plainly that there wasn't one.

> **If J shows `Unmatched` while the inner `ISNUMBER(...)` shows `TRUE`,** the two halves disagree, which means the error isn't `#N/A` at all. `ISNUMBER` returning `TRUE` proves `kw_text` and `SEARCH` are fine, so the fault is on the `kw_type` side. Put this in a spare cell to see the underlying error:
>
> ```
> =XLOOKUP(TRUE, ISNUMBER(SEARCH(kw_text, D2)), kw_type)
> ```
>
> `#VALUE!` means `kw_text` and `kw_type` are different sizes, and `XLOOKUP` requires them to match. Check with `=ROWS(kw_text) & " vs " & ROWS(kw_type)`, which must read `11 vs 11`. `#NAME?` means `kw_type` was never defined or is misspelled. `#REF!` means it points at deleted cells. All three are fixed in **Formulas → Name Manager**, not in the formula.

After filling down, filter column J for `"Unmatched"` to verify no rows are missed.

This step uses the named ranges `kw_text` and `kw_type` defined in Step 2. You can substitute the raw ranges shown there, but most of the later formulas use named ranges too, so it is a lot of extra work.

Step 4: Extract the Dose Embedded in the Order Name (Columns K and L)

Many rows have the dose written into the Order Name, such as `albumin human 5 % bottle 12.5 g`. When the Ordered Dose Amount is unusable, we pull the dose from the name before falling back to a default.

The rule: **if the name ends with a known unit, take the number immediately before it.**

Column K: Unit from Name

Add header `Name Unit` in **K1**. In **K2**:

```
=IFNA(XLOOKUP(TRUE, RIGHT(D2, LEN(name_units) + 1) = " " & name_units, name_units), "")
```

In **Google Sheets**, same change as Step 3. Wrap the comparison so all 7 units are tested rather than only the first:

```
=IFNA(XLOOKUP(TRUE,
  ARRAYFORMULA(RIGHT(D2, LEN(name_units) + 1) = " " & name_units), name_units), ""
)
```

Piece by piece:

Piece	On its own	What it contributes here
`LEN(name_units)`	Returns the character length of a string.	Across the 7-cell list this yields 7 lengths (`13, 11, 5, 3, 2, 2, 1`), one sized to each candidate unit.
`... + 1`	Adds one.	Accounts for the space that we are about to require in front of each unit.
`RIGHT(D2, ...)`	Takes N characters from the end of a string.	Handed 7 different lengths, it returns 7 different tails of the Order Name, each cut to the size of the unit it will be compared against.
`" " & name_units`	Joins text together.	Builds the 7 targets to compare against: `" VWF:RCo Units"`, `" Int'l Units"`, `" Units"`, and so on.
`<tails> = <targets>`	Compares two values. String comparison in Excel is case-insensitive.	Compares the two 7-item lists position by position, answering *does the name end with a space and then this unit?* seven times over.
`XLOOKUP(TRUE, ..., name_units)`	Returns the entry aligned with the first match found.	Returns the first unit that matched, which, given Table 4's ordering, is always the longest one that fits.
`IFNA(..., "")`	Supplies a fallback on `#N/A` only; other errors pass through.	No unit at the end produces `#N/A`; blank is the signal to column L and column M that this name carries no dose. As in Step 3, a broken `name_units` range stays visible instead of looking like "no unit found".

Together: it asks all seven "does the name end this way?" questions in a single pass and keeps the first yes. Two details in that arrangement matter:

- **Testing `" " & unit` rather than the unit alone** prevents a name ending in an ordinary word from being read as a unit. `injection` does not end with `" g"`, so it correctly returns blank.
- **The longest-first ordering of Table 4** is what makes `VWF:RCo Units` resolve as one unit instead of collapsing to `Units`.

Column L: Amount from Name

Add header `Name Amount` in **L1**. In **L2**:

```
=IF(K2 = "", "",
  VALUE(SUBSTITUTE(TEXTAFTER(TRIM(TEXTBEFORE(D2, " " & K2, -1, 1)), " ", -1), ",", ""))
)
```

Google Sheets has no `TEXTBEFORE` or `TEXTAFTER` formulas, but it does have formulas that can use regular expressions, which makes the whole extraction a single step:

```
=IFNA(VALUE(SUBSTITUTE(REGEXEXTRACT(D2, "([\d,.]+) +(?:" & TEXTJOIN("|", TRUE, name_units) &
")$"), ",", "")), "")
```

Piece by piece (Excel version):

Piece	On its own	What it contributes here
`IF(K2 = "", "", ...)`	Returns one value or another depending on a test.	A guard. No unit means no dose in the name, so it stops here and leaves the cell blank rather than letting the parsing run on a name with nothing to parse.
`" " & K2`	Joins text together.	Rebuilds the exact string column K matched on, space included, so the split happens at the unit and not at some earlier coincidence.
`TEXTBEFORE(D2, ..., -1, 1)`	Returns everything before a delimiter. `-1` means the **last** occurrence rather than the first; the trailing `1` makes the match case-insensitive.	Cuts the unit off the end, leaving the name up to and including the number. `-1` matters when the unit's letters also appear earlier in the name.
`TRIM(...)`	Removes leading, trailing, and repeated spaces.	Guarantees the last word really is the last word, so the next step can't come back empty on a stray trailing space.
`TEXTAFTER(..., " ", -1)`	Returns everything after a delimiter, here after the last space.	Takes the final token of what's left, which is the number.
`SUBSTITUTE(..., ",", "")`	Replaces every occurrence of one string with another.	Strips thousands separators, so `3,495` survives the next step instead of failing on the comma.
`VALUE(...)`	Converts text that looks like a number into an actual number.	Without it the result is the *text* `"3495"`, which cannot be converted mathematically (since it "isn't a number").

Together: the pair of `TEXT…` functions does the work, and neither half means much alone. `TEXTBEFORE` cuts away the unit on the right, `TEXTAFTER` then cuts away everything to the left of the number, and what survives both cuts is the dose. The rest is cleanup so the survivor is a number Excel can do arithmetic on.

Worked example. For `D2 = antihemophilic factor-vwf (AlphaNATE/VWF) injection 3,495 VWF:RCo Units`, where column K has resolved `VWF:RCo Units`:

```
TEXTBEFORE(D2, " VWF:RCo Units", -1, 1)  ->  antihemophilic factor-vwf (AlphaNATE/VWF)
injection 3,495
TRIM(...)                                ->  (unchanged, no stray spaces)
TEXTAFTER(..., " ", -1)                  ->  3,495
SUBSTITUTE(..., ",", "")                 ->  3495
VALUE(...)                               ->  3495      <- a number, not text
```

Note what the two cuts protect against. Taking only the last word of the name would leave `VWF:RCo` sitting in front of the number, and `VALUE("VWF:RCo")` is `#VALUE!`. Cutting at the matched unit first is what avoids that.

If the Order Name has no dose (`albumin human 5 % bottle`), K2 is blank and L2 is blank too, which is what we want, so the row falls through to the default.

> **Balfaxar is the deliberate exception.** Its names put volume last (`... 1,000 Units 40 mL infusion`), so K and L come back blank. That is fine: every Balfaxar row has a usable Ordered Dose Amount, so it resolves at step 1 and never reaches name extraction.

Step 5: Resolve the Dose (Columns M, N, O)

This is the four-step fallback. For each row, the dose is resolved by priority:

1. **Ordered Dose Amount** (E), if present *and* its unit (F) has a conversion factor for this drug.
2. **Dose from Order Name** (K and L), if step 1 didn't resolve.
3. **Administered Dose Amount** (H and I), only where the drug's `useadmin` is `yes`, the value is non-blank and **not zero**, and its unit converts. This is the last-resort evidence that something was pulled.
4. **Default dose** from the `Reference` sheet, if nothing above resolved.

The order is the stock-draw rule made literal: what was ordered outranks what was given, so a partial or zero administration can never shrink a total.

> **Why zero is skipped at step 3.** A charted `0` means the dose never reached the patient. It says nothing about stock, which was drawn regardless, so letting a `0` through would wipe out a real draw. Those rows fall through to the default instead. In the current data 10 albumin rows chart exactly this way.

Rather than re-deriving that decision in every column, we compute it **once**, in column M.

Column M: Dose Source

Add header `Dose Source` in **M1**. In **M2**:

```
=IF(AND(E2 <> "", ISNUMBER(XMATCH(J2 & "|" & F2, conv_key))), "Ordered",
 IF(K2 <> "", "Name",
 IF(AND(XLOOKUP(J2, def_type, def_admin, "no") = "yes",
        H2 <> "", H2 <> 0,
        ISNUMBER(XMATCH(J2 & "|" & I2, conv_key))), "Administered",
 IF(J2 <> "Unmatched", "Default",
 ""))))
```

Piece by piece:

Piece	On its own	What it contributes here
`E2 <> ""`	Tests whether a cell is non-blank.	Step 1 of the fallback needs an ordered amount to exist at all. This is the cheap half of that test.

Piece	On its own	What it contributes here
`J2` and `F2` joined by `&`	Joins text together.	Builds the same style of composite key as Table 2, this row's drug type and its ordered unit, pipe-separated, so a single lookup can ask about a drug *and* a unit at once.
`XMATCH(..., conv_key)`	Returns the **position** of a value in a range, or `#N/A` if it isn't there.	Asks whether Table 2 has a conversion factor for this exact drug-and-unit pair. We never use the position itself, only whether one was found.
`ISNUMBER(...)`	`TRUE` for a number, `FALSE` for anything else including errors.	Turns "found at position 8" and "`#N/A`" into a clean `TRUE` / `FALSE`, and stops the `#N/A` escaping into the rest of the formula.
`AND(a, b)`	`TRUE` only when both tests pass.	The ordered dose is usable only if it exists **and** is convertible. Either one alone is not enough.
`IF(...) IF(...) IF(...)`	Each returns one value or another on a test.	Nesting them makes the priority explicit: the first test that passes wins, so a row that could resolve two ways always takes the higher-quality one.
`XLOOKUP(J2, def_type, def_admin, "no")`	Finds this drug in Table 3 and returns its `useadmin` flag; the fourth argument is what to return when the drug isn't listed.	Keeps the albumin exception in a table rather than the formula. Defaulting to `"no"` also means an `Unmatched` row can never opt itself into step 3.
`H2 <> "", H2 <> 0`	Two ordinary comparisons.	Step 3 needs a real administered figure. The `<> 0` half is the one that matters: a charted zero is a fact about the patient, not about stock.
the final `""`	The else of last resort.	An `Unmatched` row has no drug type, so no default applies and the row correctly resolves to nothing.

Together: it is a priority chain, and the order of the four tests *is* the four-step fallback from the top of this step. The result is a plain label (`Ordered`, `Name`, `Administered`, `Default`, or blank) recording which branch the row took.

`XMATCH` works against the precomputed `conv_key` column, so this is an ordinary lookup against ordinary cells. Because column G already holds `drugtype|unit` as literal text, there is nothing to array-enter.

This column is worth its width. It makes the fallback visible while you scroll, it is filterable when a number looks wrong, and Step 7 counts it to show how much of the total rests on defaults rather than real data.

Column N: Resolved Unit

Add header `Resolved Unit` in **N1**. In **N2**:

```
=IFS(M2 = "Ordered",      F2,
     M2 = "Name",         K2,
     M2 = "Administered", I2,
     M2 = "Default",      XLOOKUP(J2, def_type, def_unit),
     TRUE,                "")
```

Column O: Resolved Amount

Add header `Resolved Amount` in **O1**. In **O2**:

```
=IFS(M2 = "Ordered",      E2,
     M2 = "Name",         L2,
     M2 = "Administered", H2,
     M2 = "Default",      XLOOKUP(J2, def_type, def_amt),
     TRUE,                "")
```

Piece by piece:

Piece	On its own	What it contributes here
`IFS(test1, val1, test2, val2, ...)`	Walks the pairs left to right and returns the value beside the first test that is `TRUE`.	Gives a flat list rather than nested logic. Because column M already decided, these are simple equality tests.
`M2 = "Ordered"` → `F2` / `E2`	Compares a cell to text.	Reads back M's label and takes the ordered unit and amount straight from the source columns.
`M2 = "Name"` → `K2` / `L2`	Same.	Takes the values that Step 4 parsed out of the Order Name.
`M2 = "Administered"` → `I2` / `H2`	Same.	Reads the administered columns, the only place in the guide that does. Reached only when column M already cleared it at step 3.
`M2 = "Default"` → `XLOOKUP(J2, def_type, def_unit)` / `def_amt`	Finds the drug type in one column of Table 3 and returns the entry beside it.	Pulls the fallback dose. N and O differ *only* here, in which column of Table 3 they read.
`TRUE, ""`	A test that always passes, so its value is always reached if nothing earlier matched.	The catch-all. Without it, `IFS` returns `#N/A` when no test matches, so unmatched rows would show an error instead of a blank.

Together: N and O are deliberately the same shape and read straight down as the fallback steps. The decision already happened in column M; these two do no thinking of their own, they only fetch the value that decision points at.

Step 6: Convert to Target Unit (Column P)

Add header `Converted Amount` in **P1**. In **P2**:

```
=IF(O2 = "", "",
  IFNA(O2 * XLOOKUP(J2 & "|" & N2, conv_key, conv_factor), "Unit not found")
)
```

Piece by piece:

Piece	On its own	What it contributes here
`IF(O2 = "", "", ...)`	Returns one value or another on a test.	A guard. Nothing resolved means nothing to convert, so the cell stays blank instead of multiplying by an empty value and reporting a misleading `0`.
`J2` and `N2` joined by `&`	Joins text together.	The same composite key as column M, but built from the **resolved** unit in N, not the ordered unit in F. That distinction is the whole point of this step.
`XLOOKUP(..., conv_key, conv_factor)`	Finds a value in one range and returns the entry beside it in another.	Retrieves the conversion factor for this drug-and-unit pair.

Piece	On its own	What it contributes here
`O2 * ...`	Multiplies.	Applies the factor, putting every row of a given drug into one common unit so they can be summed.
`IFNA(..., "Unit not found")`	Supplies a fallback on `#N/A` only; other errors pass through.	`XLOOKUP` returns `#N/A` when the pair isn't in Table 2, and this names that problem in the cell. Other errors stay visible: a `#VALUE!` here means column O holds text rather than a number, which is a different fault needing a different fix.

Together: column N settled *which* unit this row is in; this column asks Table 2 what that unit is worth for this drug and scales the amount accordingly.

> **Why the key is built from N and not F:** a row whose ordered unit was `g/kg` fell through to the name, administered, or default branch, and its resolved unit is whatever *that* branch produced. Keying off F here would look up a unit the row is no longer expressed in. This matters most for the administered branch, where the unit in I can differ from the one in F. A NovoSeven row ordered in `mcg/kg` resolves to `mcg` from column I, and only the N-based key finds the 0.001 factor.

If a row shows `"Unit not found"`, that unit has no factor for that drug. Either add it to Table 2, or check that `conv_key` and `conv_factor` actually reach the last row of the table, or investigate the row.

Step 7: Summary Results

Add a summary on the existing data sheet. Use `COUNTIF` and `SUMIF` against the helper columns.

Dose Counts

```
=COUNTIF($J$2:$J$2000, "Albumin 5%")
```

Or as a table, with drug types in column A:

Drug Type	Dose Count	Total Amount
Albumin 5%	`=COUNTIF($J$2:$J$2000,A2)`	`=SUMIF($J$2:$J$2000,A2,$P$2:$P$2000)`
Albumin 25%	`=COUNTIF($J$2:$J$2000,A3)`	`=SUMIF($J$2:$J$2000,A3,$P$2:$P$2000)`
(etc.)		

> Column P must be purely numeric for `SUMIF` to work, so make sure no rows show `"Unit not found"` in the drug type you are summing.

If you want to do this in a new sheet, name the sheet holding the raw and added data columns `Data`. In the new sheet, the formulas are then:

```
=COUNTIF(Data!$J$2:$J$2000, "Albumin 5%")
```

Or as a table:

Drug Type	Dose Count	Total Amount
Albumin 5%	`=COUNTIF(Data!$J$2:$J$2000,A2)`	`=SUMIF(Data!$J$2:$J$2000,A2,Data!$P$2:$P$2000)`
Albumin 25%	`=COUNTIF(Data!$J$2:$J$2000,A3)`	`=SUMIF(Data!$J$2:$J$2000,A3,Data!$P$2:$P$2000)`
(etc.)		

How Much of This Is Real Data?

Because column M records which branch each row took, you can audit the totals:

```
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Ordered")
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Name")
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Administered")
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Default")
```

`Ordered` and `Name` are the stock-draw measure and are the two you want to see. The other two are weaker in different ways:

- A drug whose rows are mostly `Default` has a total that reflects your assumed dose, not the record.
- A drug with any `Administered` rows has a total that is a **lower bound**, since a part-used vial still left the shelf whole.

Check both before reporting the number. For reference, the current data resolves 3,300 of 4,169 rows from `Ordered` or `Name` (79%), with 861 `Default` (all albumin) and 8 `Administered`. The `Administered` rows are concentrated in NovoSeven, where 5 of its 7 rows take that branch.

Again, if you want this in a new sheet, with the data sheet named `Data`, the formulas are:

```
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Ordered")
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Name")
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Administered")
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Default")
```

Troubleshooting

Symptom	Likely cause	Fix
J shows `Unmatched`	Order Name matches no keyword	Add a row to Table 1 (keywords), no formula edit needed
P shows `Unit not found`	The `(drug type, resolved unit)` pair has no factor	Add a row to Table 2, then fill the `key` formula down to cover it
P shows `Unit not found` for a whole drug	`conv_key` or `conv_factor` stops short of the last row of Table 2	**Formulas** → **Name Manager**, extend both to row 16
M shows `Name` when you expected `Ordered`	The unit in F is not in Table 2 for that drug (e.g. `g/kg`)	Expected for weight-based orders, they can't convert without patient weight
K is blank but the name clearly has a dose	The name doesn't **end** with the unit (Balfaxar), or the unit is missing from Table 4	Add the unit to Table 4, keeping the list longest-first

Symptom	Likely cause	Fix
L shows `#VALUE!` or `#N/A`	The token before the unit isn't a number, so the name isn't `<number> <unit>`	Inspect that Order Name; if it's a new format, it needs its own handling
A total looks far too low	Rows resolved to `Default` and the default amount is blank	Expected for the concentrates, investigate those rows rather than inventing a default
M shows `Default` when H clearly has a value	The drug's `useadmin` is `no`, or H is `0`, or I's unit has no factor for that drug	Check Table 3 column L, then whether H is a genuine zero
M shows `Administered` for an albumin row	Table 3 column L says `yes` for that albumin row	Set `useadmin` to `no`, since a partial volume understates a whole-bottle draw
A concentrate total jumped by a round number	A placeholder was typed into Table 3 rather than left blank	Clear it, placeholders feed fabricated amounts into the total
A formula returns `#NAME?`	A named range wasn't defined, or the name is misspelled	**Formulas → Name Manager** and check all nine names from Step 2
A lookup returns `#VALUE!`	The two ranges given to `XLOOKUP` are different sizes; it requires them to match	Check with `=ROWS(kw_text) & " vs " & ROWS(kw_type)`, which must read `11 vs 11`
J says `Unmatched` but `ISNUMBER` is `TRUE`	Not a matching failure at all, but a real error on the `kw_type` side that `IFNA` hid	Drop the `IFNA` wrapper to reveal the error; see the note in Step 3